

FORMATION EMBARQUÉE

TP — tp4 schneider cuve

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Gestion de cuve sous EcoStruxure + Modbus (≈ 8 h, en 3 séances)

Objectif pédagogique : refaire la démarche du TP 3 dans l'écosystème Schneider **et** ouvrir l'automate vers l'extérieur : table d'échange Modbus TCP lue par un client PC (Python ou ton programme Java du module 05). C'est le pont automatisme ↔ informatique.

 **Fiches minutées séance par séance** : [Séance 1 — Programme M221](#) · [Séance 2 — Table Modbus](#) · [Séance 3 — Supervision PC](#).

Outils

- **EcoStruxure Machine Expert – Basic** (gratuit) + son simulateur M221.
- Un client Modbus : `pip install pymodbus` (ou QModMaster en graphique).
- Prérequis : TP 3 terminé (la méthode n'est pas re-détaillée).

Cahier des charges

Une cuve alimente un procédé :

1. Mesure de **niveau analogique** 0-10 V sur `%IW0.0` (0..1000 en brut simulateur) représentant 0-100 %.
2. **Deux pompes de remplissage** P1/P2 : régulation à hystérésis — démarrage sous 40 %, arrêt au-dessus de 60 %. **Alternance** : à chaque cycle de remplissage, on utilise la pompe la moins utilisée (reprenons ton bloc du TD 08 exercice 2).
3. **Vanne de soutirage** commandée par le procédé aval (entrée TOR simulée).
4. Sécurités : **niveau très haut** (> 90 % : tout remplissage interdit, alarme), **niveau très bas** (< 5 % : soutirage interdit — protection de la pompe aval), **débordement capteur** (mesure figée > 30 s = capteur HS → défaut, positions sûres).
5. **Table d'échange Modbus TCP** pour la supervision (états, niveau, heures pompes, commandes, mot de vie).

Séance 1 (3 h) — Programme M221

Étape 1.1 — Projet et configuration

Nouveau projet Machine Expert Basic → TM221CE16R (version Ethernet). Onglet configuration : entrée analogique 0-10 V, adresse IP statique (192.168.1.50 pour la suite — le simulateur l'émule).

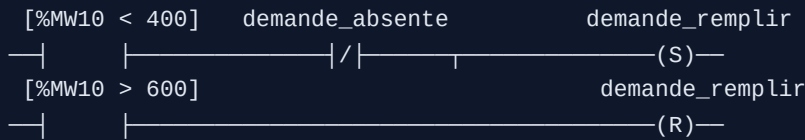
Étape 1.2 — Mise à l'échelle et symboles

- `%IW0.0` (0..1000) → `%MW10` = niveau en ‰ (0..1000 = 0..100,0 ‰) — garde les entiers ×10 (conventions du TD 08 ex. 3).

- Table de symboles complète AVANT de programmer (comme au TP 3) : `NIVEAU_POURMILLE (%MW10)` , `CMD_P1 (%Q0.0)` , `CMD_P2 (%Q0.1)` , `VANNE_SOUTIRAGE (%Q0.2)` , `DEF_CAPTEUR (%M20)` , etc.

Étape 1.3 — Les 4 fonctions, dans 4 sections nommées

1. **Régulation hystérésis** (LADDER avec blocs comparaison) :



1. **Alternance des pompes** : transcris ton bloc du TD 08 (Machine Expert Basic n'a pas de FB utilisateur : implémente en LADDER + mots `%MW30/31` pour les compteurs d'heures, ou en liste d'opérations dans une section dédiée — documente le choix).
2. **Sécurités** — section « la plus en aval », seule à écrire les `%Q` : niveau > 900 ⇒ `CMD_P1 = CMD_P2 = 0` quoi qu'il arrive ; niveau < 50 ⇒ vanne fermée ; `DEF_CAPTEUR` ⇒ tout à l'état sûr (pompes coupées, vanne fermée).
3. **Chien de garde capteur** : si `%MW10` n'a pas varié de $\pm 2\%$ pendant 30 s **alors que** une pompe tourne ou la vanne est ouverte (sinon un niveau immobile est normal !) → `DEF_CAPTEUR` . Utilise un `%TM` 30 s remis à zéro à chaque variation.

✓ **Point de contrôle 1** : au simulateur, en pilotant `%IW0.0` à la souris : hystérésis correcte (départ sous 40 %, arrêt à 60 %), alternance visible d'un cycle au suivant, sécurité niveau haut prioritaire sur tout.

Séance 2 (2 h) — Table d'échange Modbus

Étape 2.1 — Document de table (livrable clef)

Reprends le format du TD 08 exercice 3 et adapte :

Mot	Sens	Contenu	Codage
%MW100	API → PC	État : b0=P1, b1=P2, b2=vanne, b3=nivTH, b4=nivTB, b5=defCapteur	bits
%MW101	API → PC	Niveau	‰
%MW102/103	API → PC	Heures P1 / P2	h ×10
%MW104	API → PC	Mot de vie	+1/s
%MW110	PC → API	Commande : b0=autoriser remplissage, b1=acquit	bits
%MW111	PC → API	Consigne haute (arrêt pompes)	‰, borné 500..800

Sur M221, les `%MW` sont **nativement accessibles en Modbus TCP** (holding registers, adresse = numéro du mot) : il n'y a rien à « publier », mais il faut recopier explicitement états → `%MW100..104` et lire/borner

`%MW110/111` dans une section « Échange » du programme. **Jamais** le PC n'écrit directement une `%Q` : il écrit une *demande* que l'automate valide.

Étape 2.2 — Mot de vie et bornage

- `%MW104` : +1 chaque seconde (bit système `%S6` + front + bloc opération).
- `%MW111` bornée à chaque cycle : `IF < 500 THEN 500 ; IF > 800 THEN 800` — l'automate ne fait **jamais** confiance au réseau.

Séance 3 (3 h) — Client de supervision PC

Étape 3.1 — Client Python

Écris `supervision.py` (base : corrigé TD 08 ex. 3) qui, toutes les secondes : lit `%MW100..104`, affiche un tableau de bord texte, **vérifie le mot de vie**, journalise en CSV, et permet de taper `consigne 650` ou `stop` pour écrire `%MW110/111`.

```
— CUVE — 14:02:31 —
niveau   : 47,2 % ██████████
P1       : MARCHE (123,4 h)   P2 : arrêt (119,0 h)
vanne    : ouverte
défauts  : aucun             vie : OK (12043)
```

Étape 3.2 — Essais croisés (le cœur du TP)

Déroule et documente ces scénarios de bout en bout :

#	Scénario	Attendu
1	Cycle normal observé depuis le PC	niveau qui oscille 40 ↔ 60, alternance visible dans les heures
2	<code>consigne 650</code> depuis le PC	l'arrêt des pompes passe à 65 % ; <code>consigne 950</code> → borné à 80 %
3	Arrêt du simulateur automate	le PC détecte le mot de vie figé < 3 s et l'affiche en alarme
4	<code>stop</code> depuis le PC pendant un remplissage	pompes coupées PAR L'AUTOMATE (le PC n'a écrit qu'une demande)
5	Capteur figé (bloque <code>%IW0.0</code>) pendant une pompe en marche	<code>DEF_CAPTEUR</code> après 30 s, visible côté PC, positions sûres

✅ **Point de contrôle final** : scénario 3 — c'est lui qui distingue une « démo » d'une supervision : un lien TCP ouvert ne prouve pas que l'automate vit.

Étape 3.3 — (option) Version Java

Remplace le client Python par ton programme Java du module 05 §7.1 (j2mod) : même table, même comportement. Constate que la table d'échange documentée rend le langage du client **indifférent** — c'est

exactement son rôle.

Livrables et barème

Livrable	Points
Programme M221 : hystérésis + alternance + chien de garde capteur	/6
Sécurités en aval, prioritaires, testées	/3
Table d'échange documentée, bornage, mot de vie	/4
Client PC complet (lecture, écriture, vie, CSV)	/4
Les 5 essais croisés documentés	/3
Total	/20

Transfert : tu as maintenant vu les deux écosystèmes (TIA et EcoStruxure) sur la même méthode, et construit une chaîne automate ↔ supervision complète — le profil « automaticien qui parle aussi informatique » est très recherché.